

FACEBLUR — Head Model Fine-Tuning Pipeline (complete walkthrough)

This is the full, end-to-end procedure for training your own head detector and shipping it as `head.pt` for FACEBLUR. It supersedes the shorter `TRAINING_GUIDE.md` and references the three helper scripts:

- `sample_frames.py` — pull diverse frames from clips
- `bootstrap_labels.py` — pre-label them so you correct instead of draw
- `split_dataset.py` — arrange labels into the train/val layout + `data.yaml`

Why fine-tune at all: no public head model is trained on helmeted, motion- blurred, side/back, or cut-off heads (CQB / body-cam). The only reliable fix is training on frames from your *own* footage. The app already covers anyone with a visible torso via the person→head method; a fine-tuned `head.pt` adds the hard cases (heads with no body in frame) and is **unioned** in, so it can only add coverage, never remove any.

Effort reality check: the labeling is ~90% of the work; everything else is a command. Plan for 2-3 rounds of train → find misses → label misses → retrain.

Pipeline at a glance

```
clips —sample_frames.py→ dataset_raw/ (frames)
      —bootstrap_labels.py→ dataset_raw/ (frames + draft .txt)
      —[correct in Yolo_Label]→ dataset_raw/ (frames + final .txt)
      —split_dataset.py→ head_dataset/ (train/val + data.yaml)
      —yolo detect train→ runs/.../best.pt
      —rename→ head.pt → next to FACEBLUR.exe
```

What you'll produce

```
best.pt → rename to head.pt → place next to FACEBLUR.exe
```

A single-class (`head`) YOLO detector trained on frames that look like your real footage.

Step 0 — Decide scope

Goal	Labeled frames	Result
Proof-of-life (first run)	150-300	Catches easy heads; misses hard ones
Solid first model	500-800	Good on footage like what you labeled
Strong	1,500+	Robust across clips/lighting

Pull frames from **several different clips**, and keep **at least one whole clip aside that you never label** — that untouched clip is your only honest test of real-world performance (Step 7). Diversity (lighting, distance, head pose, back- of-head, helmets, blur) matters more than raw frame count.

Step 1 — Create the training environment

Training needs torch present, and a **GPU build** for any reasonable speed. Use a dedicated env so your delicate FACEBLUR build env stays untouched.

```
conda create -n yolotrain python=3.10 -y
conda activate yolotrain

REM GPU torch FIRST so ultralytics doesn't pull the CPU build:
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
REM      (use ../whl/cu118 if your Nvidia driver is older)

REM ultralytics brings cv2, numpy, pillow, pyyaml, matplotlib, etc.:
pip install ultralytics "numpy<2"
```

Verify:

```
python -c "import torch, ultralytics, cv2, numpy; print('torch', torch.__version__, '| CUDA', torch.cuda.is_available(), '| np', numpy.__version__)"
```

CUDA True = ready. CUDA False = it'll train on CPU (slow); try the cu118 index instead. That's the complete module list — the three explicit installs (torch , torchvision , ultralytics + the numpy<2 pin) pull everything else.

Step 2 — Sample frames to label

```
conda activate yolotrain
python sample_frames.py clip1.mp4 clip2.mp4 clip3.mp4 --out dataset_raw --per-clip 80
```

Saves diverse, de-duplicated frames to dataset_raw/ . Knobs:

Flag	Default	Meaning
--per-clip N	80	Aim for ~N frames per clip, spread across its length
--every N	off	Instead, keep 1 frame every N (overrides --per-clip)
--diff D	0.12	Skip frames less than D different from the last kept (0–1)
--min-blur B	0	Drop frames blurrier than B (0 = keep all, including blur — you want some)

Deliberately keep hard frames: motion blur, backs of heads, helmets, partial/ cut-off heads. Those are the high-value cases.

Step 3 — Get Yolo_Label

Download the pre-built Windows binary from the developer0hye/Yolo_Label GitHub releases page and unzip it. No compiling. It's a local desktop tool — your footage never leaves your machine.

Step 4 — Bootstrap draft labels (so you correct, not draw)

Run a model you already have over the frames; it writes a YOLO .txt next to each image plus classes.txt , which Yolo_Label loads as existing boxes.

```
python bootstrap_labels.py dataset_raw
```

Flag	Default	Meaning
--mode person	person	COCO person → head region (best recall; boxes anyone with a torso)
--mode head	—	Run a head model directly (--model head.pt); tighter, misses hard cases
--model FILE	auto	Override the model file
--conf C	0.20	Confidence floor

--mode person is the better bootstrap because it catches more (including blurred/ side/back heads of anyone with a visible body).

These drafts are a FIRST PASS, not truth. The person/head models systematically miss helmeted backs-of-heads — exactly what you're training to fix — so in Step 5 you must add every head they skipped. Trusting the drafts blindly bakes the same blind spot into your data.

Step 5 — Correct & complete labels in Yolo_Label

Open `dataset_raw/` in Yolo_Label. Define exactly **one class**: `head` (id `0`).

Labeling rules that make or break the model:

- Box the **whole head**: helmet/hair top down to chin/jaw, ear to ear.
- Label **every** head — front, **side**, **back**, helmeted, blurred, and **partial/cut-off** (box the visible part, right to the frame edge).
- If you genuinely can't tell a blob is a head, skip it — consistency beats catching every pixel. Slightly generous boxes are fine (the app pads anyway); wildly loose boxes teach the model to fire on walls.
- Keep some head-free frames with empty labels — useful negatives.

Saving: Yolo_Label writes `<image>.txt` automatically as you navigate between frames. There is no separate export step — when you're done labeling, the labels already sit next to the images.

YOLO label format (what's in each .txt)

One line per box, all coordinates normalized 0–1:

```
0 x_center y_center width height
```

The leading `0` is the `head` class. An image with no heads has an empty `.txt`.

Step 6 — Split into train/val

```
python split_dataset.py dataset_raw
```

Creates:

```
head_dataset/
├─ images/train , images/val
├─ labels/train , labels/val
└─ data.yaml          (absolute path, single class 'head')
```

Flag	Default	Meaning
<code>--out DIR</code>	head_dataset	Output dataset root
<code>--val F</code>	0.2	Validation fraction
<code>--seed N</code>	0	Random seed

It splits **by clip** (whole clips go to train or val together) so near-identical frames never leak across the split — the #1 cause of great metrics and bad real results. With only one clip it falls back to a random per-frame split and warns you. It prints the exact train command when done.

`data.yaml` it writes:

```
path: C:/.../head_dataset
train: images/train
val: images/val
names:
  0: head
```

Step 7 — Train

```
conda activate yolotrain
yolo detect train model=yolo11n.pt data=head_dataset/data.yaml epochs=100 imgsz=960 batch=8 patience=30 name=head_v1
```

Key flags:

Flag	Why
<code>model=yolol1n.pt</code>	Start from COCO-pretrained nano (fast, matches the app stack). Use <code>yolo11s.pt</code> if recall is short.
<code>imgsz=960</code>	Biggest lever for small/blurry heads. Raise to 1280 if VRAM allows.
<code>batch=8</code>	Drop to 4 on out-of-memory; raise on a big GPU.
<code>epochs=100</code> + <code>patience=30</code>	Ceiling + early stop when val stops improving.

Default augmentation (mosaic, flips, HSV) already helps pose/blur variation — no need to tune it for a first model. Output lands in `runs/detect/head_v1/`; the weights are `runs/detect/head_v1/weights/best.pt`.

Out-of-memory? Lower `batch` first (8→4→2), then `imgsz` (960→640).

Step 8 — Evaluate honestly

Look at the metrics ultralytics prints in `runs/detect/head_v1/` — especially **recall** (you care about not *missing* heads more than the odd false box; the app pads and you can over-cover for privacy). Then test on the clip you **held aside and never labeled** — the only honest measure:

```
copy runs\detect\head_v1\weights\best.pt head.pt
python test_head.py path\to\held_aside_clip.mp4
```

Open the saved `_headNN.jpg` frames and look specifically at backs-of-heads and blurred heads. Counts alone lie — look at the pictures.

Step 9 — Iterate (where the quality comes from)

Wherever it still misses on the held-aside clip:

- `python sample_frames.py held_aside_clip.mp4 --out round2_raw --per-clip 80`
- Bootstrap + correct those frames (focus on the failure type, e.g. backs of heads).
- Copy the new frames+labels into `head_dataset/images/train` and `head_dataset/labels/train`, then retrain as `name=head_v2`.

Two or three rounds of this beats one giant first batch. It's normal — plan for it.

Step 10 — Ship it

```
runs\detect\head_vN\weights\best.pt → rename to head.pt
```

Place `head.pt` next to `FACEBLUR.exe`. To ship it to all users, add to the `[Files]` section of `installer.iss`:

```
Source: "head.pt"; DestDir: "{app}"; Flags: ignoreversion
```

How the app uses it: with **Detect whole head** on, FACEBLUR auto-detects the `head` class from the model's own metadata, runs `head.pt` at full resolution with edge strips, and **unions** its boxes with the person→head method and the face boxes. A face already covered by a head box is not censored twice. So your model only **adds** coverage. Turn on **Show debug boxes** to see it work: red outlines are `head.pt` firing, yellow is person→head region, cyan is the face model.

Full command cheat-sheet

```
REM one-time env
conda create -n yolotrain python=3.10 -y
conda activate yolotrain
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
pip install ultralytics "numpy<2"

REM per round
python sample_frames.py clip1.mp4 clip2.mp4 --out dataset_raw --per-clip 80
python bootstrap_labels.py dataset_raw
REM ... correct + add missed heads in Yolo_Label (one class: head) ...
python split_dataset.py dataset_raw
yolo detect train model=yolo11n.pt data=head_dataset/data.yaml epochs=100 imgsz=960 batch=8 patience=30 name=head_v1
copy runs\detect\head_v1\weights\best.pt head.pt
python test_head.py held_aside_clip.mp4
REM ... iterate on misses, then ship head.pt next to FACEBLUR ...
```

Troubleshooting & pitfalls

Symptom	Cause / Fix
Great val metrics, bad real footage	Train/val leakage . Split by clip (the script does), and never label your held-aside test clip.
Model fires on walls/gear	Loose/inconsistent boxes, or too few negatives. Tighten labels; keep some head-free frames.
Misses small/distant heads	<code>imgsz</code> too low. Raise to 960/1280.
Out-of-memory in training	Lower <code>batch</code> (8→4→2), then <code>imgsz</code> (960→640).
Misses backs-of-heads specifically	You didn't label enough of them. That's the whole point — over-represent them.
<code>CUDA False</code> , training crawls	Install the CUDA torch build (Step 1); try <code>cu118</code> if <code>cu121</code> mismatches your driver.
<code>split_dataset.py</code> says "no .txt"	Labels weren't saved. Yolo_Label writes <code><image>.txt</code> as you navigate; make sure you moved through the frames.
App still shows face-only with <code>head.pt</code> present	Confirm <code>head.pt</code> is next to the exe and Detect whole head is on; check the log line naming the head method.

FACEBLUR head fine-tuning pipeline — for use with `sample_frames.py`, `bootstrap_labels.py`, `split_dataset.py`.